# Manual de Implementación — OctoMatrix

Qué hacer DESPUÉS de tener el modelo exportado
Semillero BlackByte · Universidad Tecnológica de Pereira
Docente: Biviana Yulieth Aguirre Arias · Integrante: Santiago Potes Giraldo

## 0. Punto de partida

Este manual asume que ya corriste el entrenamiento (ver *informe_laboratorio.pdf* / *reproducir.sh*) y tienes en **models/** los artefactos:

| Artefacto | Qué es |
|---|---|
| models/octomatrix.joblib | experto de texto (real) + su .sha256 y _meta.json |
| models/moe/ | MoE: experto real + experto generado (ensemble.json) |
| models/flows.joblib | experto de flujos de red (opcional) |

OctoMatrix es una **capa de validación**: no reemplaza otros controles de seguridad; emite una evaluación (normal/sospechoso) y la aplicación decide la acción.

## 1. Cargar el modelo

Se carga una sola vez por proceso. La carga verifica el sha256 del artefacto antes de deserializar (integridad).

```
from octomatrix.moe import ScoreUnionEnsemble
from octomatrix.data.text import request_text

moe = ScoreUnionEnsemble.load("models/moe")   # una vez al iniciar
```

## 2. Evaluar una petición

La aplicación normaliza el evento (método, ruta, query, cuerpo) y lo pasa como texto. El ensemble devuelve la decisión (OR de los expertos) y qué experto disparó.

```
texto = request_text("POST", "/login", "",
                     "user=admin&pass;=admin' OR '1'='1")
alerta = bool(moe.predict([texto])[0])          # True = ataque
quien  = moe.attributions([texto])[0]           # ["real"], ["generated"]...
margen = float(moe.margin([texto])[0])          # cercanía a la frontera
```

Decisión según resultado: la aplicación registra, bloquea o alerta. OctoMatrix no decide la acción final.

## 3. Integración como middleware (productor/consumidor)

En una arquitectura con Kafka u otra cola, OctoMatrix va entre el productor de eventos y la lógica de negocio:

1. La aplicación (p. ej. Parchate Pereira) produce el evento HTTP.
2. Un consumidor lo normaliza con *request_text*.
3. Llama a *moe.predict* (en lote para rendimiento: ~2 ms/petición).
4. Publica la decisión; la aplicación actúa.

**Importante para Parchate Pereira:** reemplazar el *_sanitize_input* actual (una blacklist que borra <, > y 'script', evadible con <*scr<script>ipt*>) por esta evaluación del modelo. La blacklist da falsa sensación de seguridad; el detector mide probabilidad de ataque sobre el payload completo.

## 4. Rendimiento y operación

• Procesar en LOTE siempre que se pueda: ~2 ms/petición vs ~39 ms suelta.

• Cargar el modelo una sola vez por proceso (no por petición).

• No modificar el .joblib en caliente; si cambia, reiniciar el proceso.

• Registrar todas las decisiones (entra el lazo de post-entrenamiento, sección 6).

• Un umbral más estricto (FPR 0,1%) detecta menos pero molesta menos; decidir con negocio.

## 5. Experto de flujos (si se usa)

El experto de flujos (DoS, fuerza bruta) consume **flujos de red**, no peticiones HTTP. Va en paralelo, sobre la telemetría de red (CICFlowMeter), y compone a nivel de alerta con el experto de texto: hay alerta si cualquiera dispara. No se combinan por-petición porque son telemetrías de distinta granularidad.

## 6. Post-entrenamiento (lazo de feedback)

El modelo NO se reentrena solo. El tráfico de producción se registra, se seleccionan los casos dudosos (margen pequeño) para revisión humana, y lo revisado alimenta un reentrenamiento **manual**. El lazo se rompe a propósito en la revisión humana: ahí es donde entraría el envenenamiento de datos si fuera automático.

```
python scripts/postprod.py log   --model models/moe --traffic trafico.jsonl
python scripts/postprod.py queue --band 0.1 --max 300
# una persona revisa data/feedback/review_queue.jsonl (label 0/1 + reviewer)
python scripts/postprod.py ingest
```

Para cerrar la brecha de generalización, alimentar el experto real con tráfico benigno de tus propias apps (access logs) vía *--own-logs* (ver docs/BENIGN.md).

## 7. Limitaciones conocidas

• Depende de la calidad del entrenamiento; no detecta ataques completamente nuevos.

• No generaliza aún a sitios no vistos: necesita benigno propio.

• No reemplaza validaciones determinísticas ni un WAF de reglas.

• IDOR y escalada de privilegios no se detectan bien sólo por el payload.

• La verificación sha256 protege contra corrupción, no contra un atacante que controle ambos archivos.